Backup and Recovery Module Reviewer

Code Review and Technical Documentation Guide

This reviewer explains the implemented Backup and Recovery module of GrocerEaseIMS. It is written for student presentation, defense preparation, and technical review.

The document focuses only on the implemented manual features: Create Backup, Restore Backup, View Backup History, and Download Backup. Delete Backup is discussed as a requested feature that is not implemented in the current backup code.

Automatic Backup is intentionally not included in this reviewer because it is outside the requested scope.

Backup and Recovery Module Reviewer

GrocerEase Inventory Management System

1. Module Overview

The Backup and Recovery module protects the system database by creating full SQL backup files and allowing an administrator to restore data from a selected SQL backup. In simple terms, it gives the administrator a way to save the current database state and recover it later if records are damaged, deleted, or changed incorrectly.

In this system, backups are stored as .sql files inside the local /backups folder. The module is important because GrocerEaseIMS manages products, categories, suppliers, stocks, and transaction-related records. If the database becomes corrupted or important data is accidentally changed, a valid backup can help bring the system back to an earlier working state.

The implemented backup module is mainly composed of a frontend page in views/inventory/settings.php, a backend controller in controllers/backup.php, and a reusable model class in models/BackupManager.php.

2. Features Covered

Create Backup

- Purpose: Creates a full SQL copy of the current MySQL database.
- User/Admin action: Admin clicks Backup now and confirms the SweetAlert prompt.
- System action: The frontend sends POST controllers/backup.php?action=backup. The controller calls BackupManager::createBackup().
- Expected result: A file named backup_YYYY-MM-DD_HH-MM-SS.sql is created in /backups and the backup history refreshes.
- Possible errors: Expired session, invalid request method, unwritable backup folder, no database tables found, invalid generated SQL, or unexpected server response.

Restore Backup

- Purpose: Replaces the current database content by executing SQL statements from a valid backup file.
- User/Admin action: Admin either uploads a .sql file or selects a valid backup from history and confirms restore.
- System action: The selected file is validated, a pre_restore safety backup is created, foreign key checks are disabled, SQL statements are executed, and foreign key checks are enabled again.
- Expected result: The database is restored and the system shows the safety backup filename.
- Possible errors: Invalid file, unreadable or empty backup, file does not look like SQL, safety backup creation fails, SQL execution fails, or server returns non-JSON output.

View Backup History

- Purpose: Shows available local backup files so the admin can inspect, download, or

restore them.
- User/Admin action: Admin opens the Settings backup page.
- System action: JavaScript calls GET controllers/backup.php?action=list. BackupManager scans /backups/*.sql files and returns filename, date, type, size, readable size, and valid status.
- Expected result: The table displays backup files sorted by newest date first.
- Possible errors: Controller path is wrong, session expired, JSON parsing fails, backup folder cannot be scanned, or no files exist.

Download Backup

- Purpose: Allows the admin to download an existing SQL backup file.
- User/Admin action: Admin clicks the download icon in the backup history table.
- System action: Browser navigates to controllers/backup.php?action=download&file=filename. The controller resolves the safe path and streams the SQL file.
- Expected result: The selected .sql file is downloaded.
- Possible errors: File is missing, filename is invalid, path validation fails, session expired, or the controller returns 404 File not found.

Delete Backup

- Purpose requested: Remove an old or unwanted backup file from storage.
- Actual current status: Not implemented in the backup module code reviewed. There is no delete action in controllers/backup.php, no deleteBackup method in BackupManager.php, and no delete button in views/inventory/settings.php.
- Expected result if implemented later: Admin confirms delete, controller validates filename inside /backups, deletes the file with unlink(), returns JSON, and refreshes history.
- Possible errors if implemented later: Invalid filename, missing file, permission denied, attempt to delete outside /backups, or unauthorized request.

3. Workflow Explanation

The flow starts when the administrator opens the Backup and Recovery area in the Settings page. The page displays backup status cards, action buttons, restore file controls, and a backup history table. The admin then chooses an action: create backup, restore backup, view history, or download backup. Delete backup should only be shown in the flow as not implemented unless the code is updated.

Backup Flow

1. Admin clicks Backup now.
2. System shows confirmation asking if the admin wants to create a database backup.
3. If cancelled, the flow ends with no change.
4. If confirmed, the frontend sends POST controllers/backup.php?action=backup.
5. The controller checks if the admin session is valid and if the request method is POST.
6. BackupManager exports the full database into a SQL file.

7. The generated SQL file is validated.
8. If backup creation fails, the UI shows an error message.
9. If successful, the SQL file is saved in /backups and history is refreshed.

Recovery Flow

1. Admin chooses a backup source: upload a .sql file or restore an existing file from backup history.
2. The system validates the selected file extension, readability, file size, and SQL-like content.
3. If invalid, the system shows a validation error.
4. If valid, the admin confirms the restore warning.
5. If cancelled, restore stops.
6. If confirmed, the system creates a pre_restore safety backup first.
7. The system reads and splits SQL statements, disables foreign key checks, executes the restore, and enables foreign key checks again.
8. If restore fails, the system shows an error and includes the safety backup filename when available.
9. If restore succeeds, the system shows a success message and refreshes backup history.

Backup History Flow

1. Settings page calls GET controllers/backup.php?action=list.
2. BackupManager scans the local /backups folder for .sql files.
3. Each file is checked for filename, date, type, size, and valid status.
4. The frontend displays rows with Download and Restore buttons.
5. Download streams the file through the backup controller.
6. Delete is not part of the current backup history UI.

Decision Box Meanings

- Admin chooses backup and recovery action: User chooses backup, restore, history, download, or requested delete.
- Is the database connection valid: The PDO connection must be available from config/db.php and autoload.php.
- Was the backup created successfully: BackupManager must create and validate a non-empty SQL file.
- Does the admin confirm restore: SweetAlert confirmation must be accepted before restore continues.
- Is the selected backup file valid: The file must be .sql, readable, non-empty, and SQL-like.
- Was the restore completed successfully: SQL statements must execute without exception.
- Admin chooses backup history action: Current code supports download and restore; delete is not implemented.

4. Code Review

File	Purpose	Important Functions

Backup and Recovery Module Reviewer

GrocerEase Inventory Management System

---

models/BackupManager.php	Core backup logic. Creates SQL backups, restores SQL, validates files, lists backup history, resolves safe paths.	createBackup, restoreBackup, getBackupList, resolveBackupPath, validateBackupFile, dumpTable, splitSqlStatements
controllers/backup.php	Authenticated HTTP endpoint for backup module actions. Returns JSON for most actions and streams SQL for download.	respondJson, requireLogin, requireMethod, switch actions: backup, restore, restore_specific, validate, list, download
views/inventory/settings.p	Frontend UI for backup controls, restore upload, backup history, download, and restore from history.	backupButton click handler, restore file validation, loadBackupHistory, downloadBackup, restoreSpecific
views/inventory/dashboard.	Contains a quick action that can trigger backup creation from the dashboard.	backupFromDashboard
autoload.php	Starts session, loads database config, registers model autoloading, creates PDO connection.	spl_autoload_register, DB connection creation
config/db.php	Defines MySQL connection credentials and creates a PDO connection.	DB::GetConnection

Data moves from frontend to backend through fetch requests. For backup and list, the action can be in the query string. For restore and validate, FormData is used because files or filenames are sent through POST. The controller returns JSON objects with status and message. The frontend reads data.status and shows SweetAlert success or error messages.

5. How the Code Works

Create Backup Technical Process

1. Admin clicks Backup now in settings.php or uses the dashboard quick action.
2. JavaScript shows SweetAlert confirmation.
3. If confirmed, JavaScript sends POST ../../controllers/backup.php?action=backup.
4. controllers/backup.php requires login and POST method.
5. The controller calls BackupManager::createBackup().
6. createBackup builds a timestamped filename and opens it in /backups.
7. It reads all base tables using SHOW FULL TABLES WHERE Table_type = BASE TABLE.
8. For each table it writes DROP TABLE IF EXISTS, CREATE TABLE SQL, and INSERT INTO rows.
9. It writes SQL settings including SET FOREIGN_KEY_CHECKS=0 at the start and SET FOREIGN_KEY_CHECKS=1 at the end.
10. It validates the generated SQL file and returns JSON status, message, filename, size, and table count.

11. The frontend shows Backup created or Backup failed and refreshes backup history on success.

Restore Backup Technical Process

1. Available backup files are loaded by loadBackupHistory through action=list.
2. Admin can upload a .sql file or restore a valid file from the backup history table.
3. Uploaded files are checked first by extension in JavaScript, then by action=validate in the controller.
4. BackupManager::validateBackupFile checks existence, .sql extension, readability, non-empty file size, and SQL-like content.
5. Before restore, SweetAlert warns the admin that current database state will be replaced.
6. For uploaded restore, action=restore sends backup_file through FormData.
7. For history restore, action=restore_specific sends the selected filename.
8. The controller resolves stored filenames safely through BackupManager::resolveBackupPath.
9. BackupManager::restoreBackup validates the file again, creates a pre_restore safety backup, reads SQL content, splits statements, disables foreign key checks, executes each statement, and re-enables foreign key checks.
10. On success, JSON includes status success and safety_backup. On failure, JSON includes an error message and the safety backup filename when it was created.

Backup History, Download, and Delete Technical Process

- Backup history is not stored in a database table. It is built dynamically by scanning /backups/*.sql files.
- getBackupList returns filename, file modification date, type, raw size, readable size, and valid status.
- Download uses action=download and file=filename. The controller validates the filename path and streams the SQL file with application/sql headers.
- If a file is missing during download, the controller returns HTTP 404 and plain text File not found.
- Delete backup is not currently implemented. There is no action=delete case in controllers/backup.php and no frontend function that deletes a backup file.

6. Database Involvement

- Database used: MySQL database named grocer_easedb, connected through PDO in config/db.php.
- Tables involved: The backup process includes all MySQL base tables returned by SHOW FULL TABLES. This means products, categories, suppliers, stock, product-supplier relation tables, transaction logs, users, and other base tables are included if present in the database.
- Backup history storage: There is no database table for backup history. History is based on actual SQL files in the /backups folder.
- Filename and date handling: The filename uses a prefix and date/time format. The date displayed in history is the file modification time from filemtime().

- Status handling: Valid status is calculated each time by validating the backup file. It is not permanently stored in the database.
- File path handling: Stored backup filenames are sanitized with basename and a regex, then resolved with realpath to ensure they stay inside /backups.
- During restore: SQL statements from the backup are executed against the current MySQL database. Existing tables may be dropped and recreated based on the backup content.

7. Frontend and Backend Connection

UI Action	JavaScript	Endpoint	Response
Backup now	backupButton click	POST controllers/backup.php?action=ba	JSON success/error
Upload restore file	restoreFileInput change	POST controllers/backup.php action=validate	JSON validation result
Restore uploaded file	restoreUploadedButton click	POST controllers/backup.php action=restore	JSON success/error with safety_backup
Load history	loadBackupHistory	GET controllers/backup.php?action=li	JSON list data
Download backup	downloadBackup	GET controllers/backup.php?action=do	SQL file stream or 404
Restore from history	restoreSpecific	POST controllers/backup.php action=restore_specific	JSON success/error with safety_backup
Delete backup	Not present	Not present	Not implemented

Common Error Causes

- 404 Not Found for backup controller: wrong relative path, controller file missing, app installed in a different folder, server rewrite/path issue, or request sent from a page with a different nesting level.
- Unexpected token '<': JavaScript expected JSON but received HTML. Common causes are PHP fatal error page, login page due to expired session, 404 HTML page, or warning/notice printed before JSON.
- Backup button not working: JavaScript error, SweetAlert not loaded, wrong controller path, session expired, POST blocked, database connection failure, or /backups folder not writable.
- Restore not working: invalid .sql file, file upload error, PHP upload limit, failed safety backup, SQL statement error, permission problem, or foreign key/data incompatibility.
- History not loading: action=list fails, session expired, controller returns HTML, /backups folder missing or unreadable, JSON parse error, or JavaScript rendering error.

8. Security and Safety Review

- Admin-only access: controllers/backup.php calls requireLogin(), so actions require a logged-in session.

Backup and Recovery Module Reviewer

GrocerEase Inventory Management System

- Method restrictions: backup, restore, restore_specific, and validate require POST; list and download require GET.
- Restore file validation: files must be .sql, readable, non-empty, and must contain SQL-like markers such as CREATE TABLE, INSERT INTO, GrocerEaseIMS Backup, or MySQL dump.
- Path validation: resolveBackupPath uses basename, a strict filename regex, realpath, and a directory containment check.
- Safety backup: restore creates a pre_restore backup before applying SQL changes.
- Risk: SQL files are stored under a project folder. If the web server exposes /backups directly, users may access database dumps if they know or list filenames.
- Risk: Any valid-looking SQL file can be restored by an authenticated admin. A bad SQL file can overwrite or damage the database.
- Risk: There is no audit log table recording who created, restored, downloaded, or attempted backup actions.
- Risk: Delete backup is not implemented, so old backups must be removed manually unless another process handles cleanup.
- Recommended improvements: protect /backups with server rules, move backups outside public web root, add CSRF protection, add role-based admin checks, add audit logs, add delete with confirmation and path validation, use transactions where possible, and test restore on a staging copy before production use.

9. Problems Found and Fix Suggestions

Problem Found	Location	Why It Is A Problem	Suggested Fix	Priorit
Delete backup feature requested but not implemented	controllers/backup.p BackupManager.php, settings.php	The reviewer/flow includes Delete Backup, but no endpoint, model method, or UI button exists.	Add deleteBackup method, action=delete, SweetAlert confirmation, POST only, path validation, and refresh history.	High
Backups may be publicly accessible	/backups folder	SQL files may contain full database data including sensitive records.	Move backups outside public web root or add server deny rules such as .htaccess or web server config.	High
No audit logging	No backup audit table/code	Cannot prove who created, restored, downloaded, or attempted backup actions.	Create backup_audit_logs table and insert action, admin id, filename, status, IP, and timestamp.	Medium
No CSRF token on backup/restore requests	settings.php and backup.php	A logged-in admin could be tricked into sending a backup/restore request.	Add CSRF token to forms/fetch requests and verify in controller.	High
Restore executes uploaded SQL	BackupManager::resto	A malicious or wrong SQL file can damage the database if accepted.	Restrict restore to trusted generated backups or add stricter signature/header checks.	High
Download returns plain text 404 instead of JSON	controllers/backup.p download action	Normal for file download, but inconsistent for AJAX-style error handling.	Keep as is for browser download, or add optional JSON error mode for	Low

			fetch-based UI.	
Backup history not stored in database	BackupManager::getBa	History depends only on files, so deleted/moved files remove history.	Optional: add backup_records table if persistent audit/history is required.	Low
Database credentials are hardcoded	config/db.php	Credentials in source code are risky if repository is shared.	Move credentials to environment config or a local ignored config file.	Medium

10. Presentation Guide

Opening Script

Good day. This part of GrocerEaseIMS is the Backup and Recovery module. Its purpose is to protect the inventory database by allowing an administrator to create a full SQL backup, restore a valid backup, view backup history, and download backup files.

Create Backup Script

When the admin clicks Backup now, the system asks for confirmation. After confirmation, the frontend sends a POST request to the backup controller. The controller calls BackupManager, which exports all database tables into a timestamped SQL file inside the /backups folder. If the SQL file is valid, the system shows a success message and refreshes the backup history.

Restore Backup Script

For restore, the admin can upload a SQL file or select a valid file from backup history. The system validates the file before allowing restore. Before changing the database, it creates a pre_restore safety backup. Then it executes the SQL backup into the MySQL database. If successful, the system shows the safety backup filename.

View History, Download, and Delete Script

Backup history is displayed by scanning the local /backups folder. The table shows filename, date, type, size, and validity status. The admin can download a backup through the controller. Delete Backup is not included in the current implemented code, so I will present it as a missing or future improvement, not as a working feature.

Automatic Backup Statement

Automatic Backup is not included in this reviewer because the requested scope is the manual Backup and Recovery module. I will not present it as part of the implemented workflow.

Panelist Answer Tips

- If asked why backup is needed: It protects the database from accidental data loss or wrong changes.
- If asked where files are stored: They are stored locally in the project /backups folder.
- If asked about delete: The current backup module has no delete endpoint yet; it is a recommended improvement.

Backup and Recovery Module Reviewer

GrocerEase Inventory Management System

- If asked about restore safety: The system creates a pre_restore safety backup before applying restore.
- If asked about security: Login is required and paths are validated, but backups should still be protected from public access.

11. Possible Questions and Answers

Question	Answer
What is the purpose of backup and recovery?	To save the current database state and restore it later if data is lost, damaged, or changed incorrectly.
Why is backup important?	The system stores important inventory records. A backup helps recover products, suppliers, stocks, and other records after mistakes or failures.
What happens when backup fails?	The controller returns an error JSON response and the frontend shows a Backup failed SweetAlert message.
What happens when restore fails?	The system returns a restore error. If the safety backup was created, the message includes its filename.
Why should restore be restricted to admin?	Restore can replace the entire database, so only trusted administrators should be allowed to do it.
Where are backup files stored?	They are stored as .sql files in the local /backups folder.
How does the system know if backup is successful?	BackupManager creates the SQL file, validates it, and returns status=success with filename, size, and table count.
Why is automatic backup not included?	It is not included in this reviewer because the requested module scope is manual Backup and Recovery only.
What are the risks of restore?	A restore can overwrite current database data. A wrong or malicious SQL file can damage records or structure.
How do you prevent invalid backup files?	The code checks file existence, .sql extension, readability, non-empty size, and SQL-like content before restore.
Is Delete Backup working?	No. Delete Backup is not implemented in the reviewed backup code. It should be added before presenting it as a working feature.
What happens if a backup file is missing during download?	The controller returns HTTP 404 with File not found.

12. Final Summary

The Backup and Recovery module is a practical protection feature for GrocerEaseIMS. The admin can create a full SQL backup, view available backup files, download them, and restore the database from a valid backup. The module uses BackupManager.php for the core database export and restore logic, controllers/backup.php for authenticated backend actions, and views/inventory/settings.php for the user interface.

The strongest safety feature is the pre_restore backup created before restore. The important limitations are that Delete Backup is not implemented, backup audit logging is missing, and the /backups folder should be protected from public access. For presentation, the module

should be explained as a manual backup and recovery workflow, not an automatic backup system.